



Prédire les retards de livraison dans la chaîne d'approvisionnement



Problème



DataCo Global est une société qui vend différents types de produits. Afin de satisfaire ses clients, elle doit assurer la préparation et la livraison des commandes dans des délais raisonnables.

L'entreprise fait face à un problème de retards dans les livraisons. Certains clients reçoivent leurs commandes après la date prévue, ce qui peut nuire à la réputation de l'entreprise.

Pour améliorer la situation, DataCo Global souhaite identifier à l'avance les livraisons qui risquent d'avoir du retard.

Nous allons étudier comment l'intelligence artificielle peut aider à résoudre cette situation ou y apporter une réelle valeur ajoutée.

C'est quoi l'IA?



- Points rouges : images de fraises
- Points bleus : images de myrtilles
- Tracer les points : Preprocessing ou travail sur les données
- Tracer la ligne verte :
 - a. Choisir le modèle
 - b. Entraîner le modèle pour choisir les bons paramètres

Data Preprocessing

Travailler les données pour pouvoir les exploiter

01

Enlever les
colonnes inutiles

02

Gérer les valeurs
nulles dans le jeu
de données

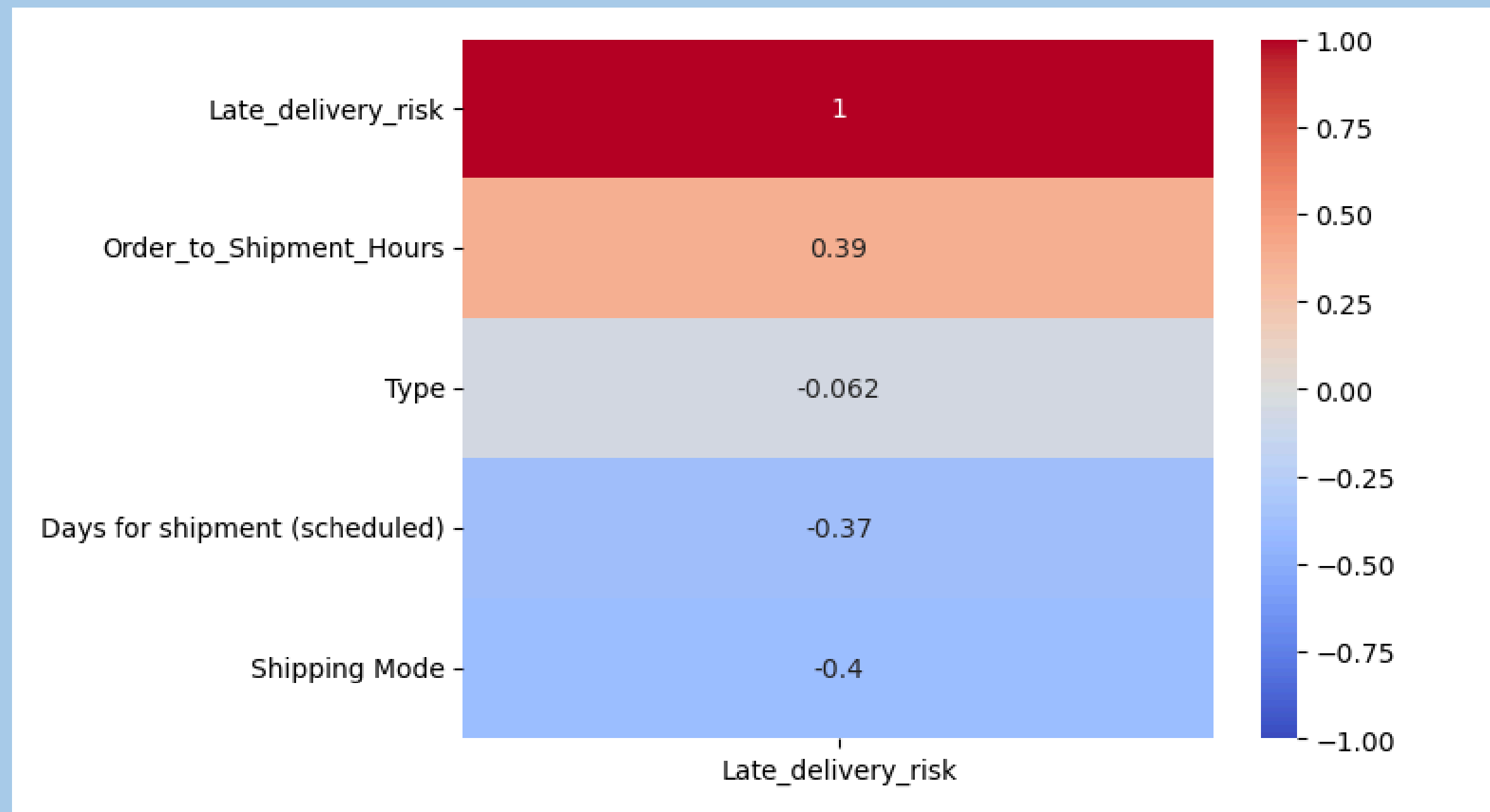
03

Créer de
nouvelles
données plus
pertinentes

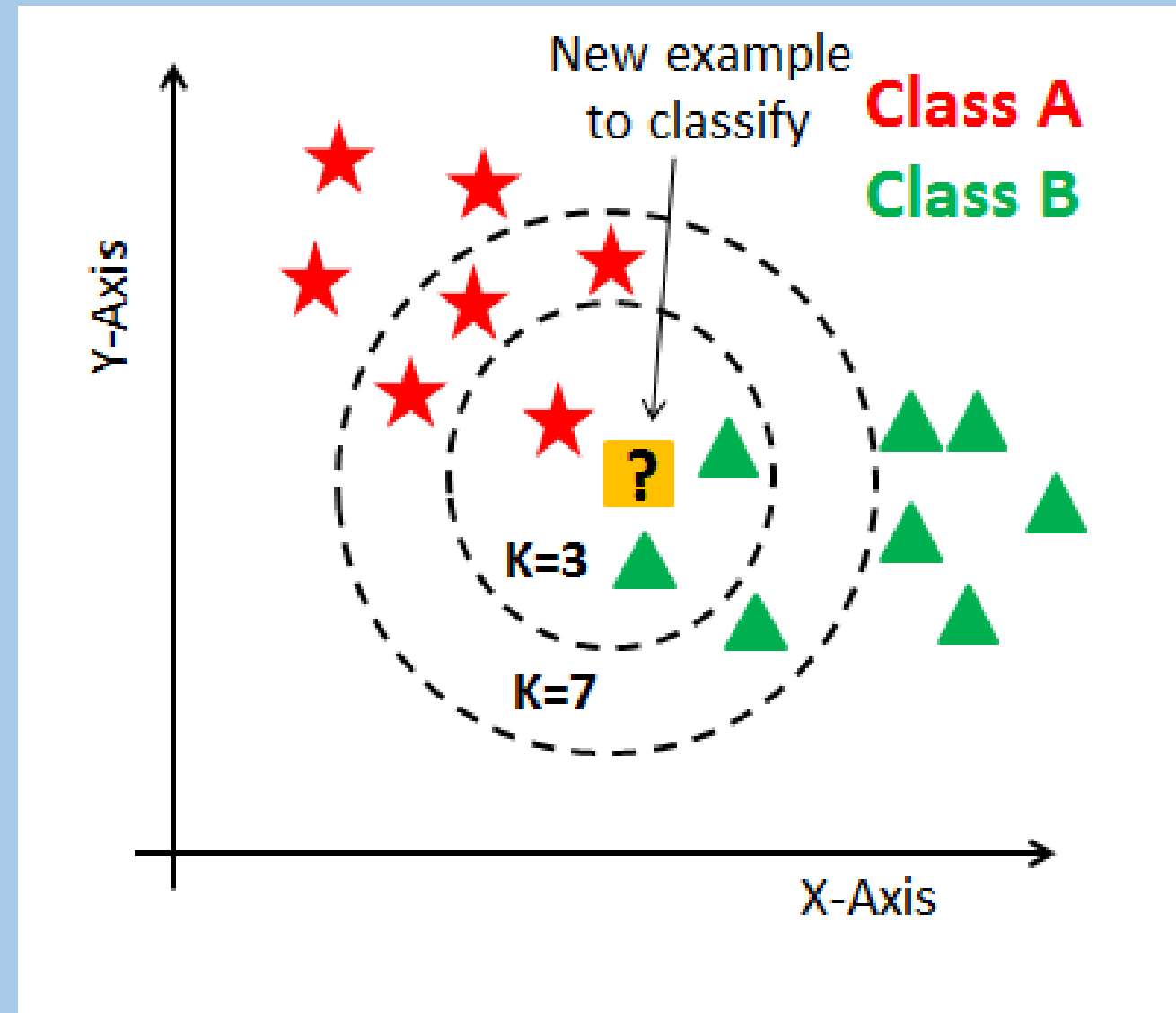
04

Convertir et
standardiser
les données

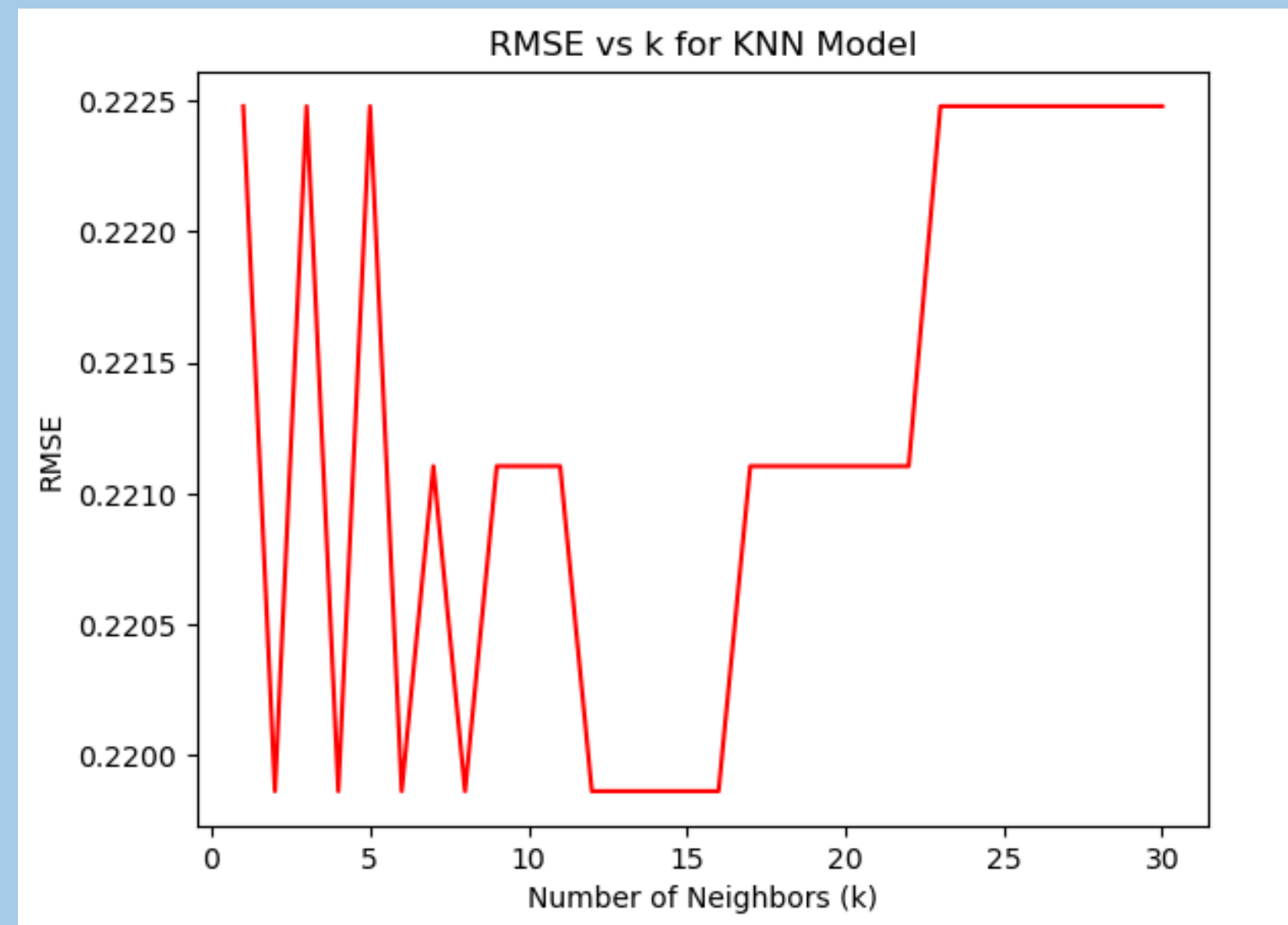
Matrice de corrélation



Méthode des k plus proches voisins



Entraînement des données



Entraînement des données

```
# Create a KNN model with 14 neighbors
model = KNeighborsClassifier(n_neighbors=14)

# Train the model using the training data
model.fit(X_train, Y_train)

# Make predictions on the training set
y_pred = model.predict(X_train)

# Compute the Root Mean Squared Error (RMSE) between true labels and predictions
rmse = np.sqrt(mean_squared_error(Y_train, y_pred))
print("RMSE:", rmse)

# Compute the classification accuracy in percentage
accuracy = accuracy_score(Y_train, y_pred) * 100
print("Taux de réussite:", accuracy.round(2), "%")
```

RMSE: 0.21986326180900173
Taux de réussite: 95.17 %

Résultats

```
# Make predictions on the test set using the model already trained on the training data
y_pred_test = model.predict(X_test)

# Compute RMSE on the test set
rmse_test = np.sqrt(mean_squared_error(Y_test, y_pred_test))
print("RMSE (test):", rmse_test)

# Compute accuracy on the test set in percentage
accuracy_test = accuracy_score(Y_test, y_pred_test) * 100
print("Taux de réussite:", accuracy_test.round(2), "%")
```

```
RMSE (test): 0.2165463255316585
Taux de réussite: 95.31 %
```

Merci pour votre attention !